Nesne Yönelimli Programlama – Konu Anlatımı

UML: Sınıf İlişkileri, Sequence ve Diğer Diyagramlar

Slaytlar, ders kitabı (Schach 8e), ders videoları ve WhatsApp grubu temel alınmıştır

9 Kasım 2026

8 UML: Sınıf İlişkileri, Sequence ve Diğer Diyagramlar

! Hoca derste vurguladı Hafta 9 ve 2025 vizesi

Bu konu kitabın değil, hocanın paylaştığı `1uml.pdf` sunumunun ("Introduction to UML") konusudur. Hafta 9 sonunda: *"Sınavımız bugünkü konulara kadar olan kısımları kapsıyor"* ⇒ vize kapsamındadır. *"Geçmiş yıllarda ben bununla ilgili sorular soruyordum sınavda: klas diagramını vererek hem sınıfların arasındaki ilişkileri yorumlamanızı hem de sayıları yorumlamanızı istiyordum. Yani sadece bu bir binary association falan diyerek bırakmamalıyız."* 2025 WhatsApp: *"uml pdf'inin 35. sayfasında bir class example var; böyle bir class diyagramı verip aralarındaki ilişkileri yorumlamanızı isteyebilirim ama detaylı isterim."* 2025–26 vizesinde: *rent a car otomasyonu, okların çeşitleri (aggregation, composition, inheritance vs.) ve işlevleri* + aynı otomasyonun *sequence diyagramı* (24 puan).

8.1 UML nedir, neden kullanılır?

- **Unified Modeling Language**: OMG (Object Management Group) standardı; Booch, Rumbaugh ve Jacobson'ın çalışmalarına dayanır (1994 Booch–Jacobson, 1995 Rumbaugh katıldı, 1997 UML 1.1, bugün UML 2.x).
- Belgeleri ve yazılımı ifade etmek ve tasarlamak için bir **modelleme dilidir**; özellikle nesne yönelimli tasarımda yararlıdır. **Bir süreç değildir** ve **programlama dilinden bağımsızdır**.
- Neden? Açık standart, yazılım sistemlerini *belirlemek, görselleştirmek, inşa etmek ve belgelemek* için grafik notasyon; ilk tasarımdan ayrıntılı tasarıma kadar bütün yaşam döngüsünde kullanılır; müşteri ve geliştiriciler arasında anlamayı ve iletişimi artırır; birçok araç destekler.
- **Model** bir sistemin bir alt kümesini tanımlayan soyutlamadır; **view** bir modelin seçilmiş yönlerini gösterir; **notation** görünümleri çizmek için grafik/metin kurallarıdır. (Örnek: sistem = uçak; modeller = uçuş simülatörü, ölçekli model; görünümler = planlar, elektrik tesisatı, yakıt sistemi.)
- UML çok diyagramlıdır; her diyagram bir modele bakan bir görünümdür. *Problemlerin %80'i UML'in %20'siyle modellenebilir.*

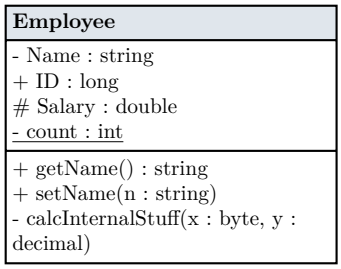
UML baseline diyagramları: use case diagrams, class diagrams, package diagrams, interaction diagrams (sequence, collaboration), activity diagrams, state transition diagrams, (component ve) deployment diagrams.

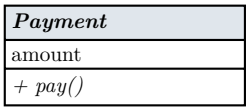
Statik ve dinamik: *Statik tasarım* kod yapısını ve nesne ilişkilerini anlatır, değişmez (class, object, package, component, deployment diyagramları). *Dinamik tasarım* nesneler arası iletişimi, olay sıralarını gösterir, senaryoya göre değişir (sequence, collaboration, activity, state diyagramları).

8.2 Class diagram

Sistemi, sınıflarını ve aralarındaki ilişkileri göstererek özetler. Class diyagramları **statiktir**: neyin neyle etkileştiğini gösterir ama etkileşince *ne olduğunu* göstermez. Her sınıfın nitelik ve işlemlerini de gösterir; sistemin genel mimarisini anlatmanın iyi bir yoludur. Üç bakış açısıyla çizilir: **conceptual** (yazılımdan ve dilden bağımsız), **specification** (yazılımın arayüzlerine odaklı), **implementation** (gerçekleştirmeye odaklı).

8.2.1 Sınıf notasyonu



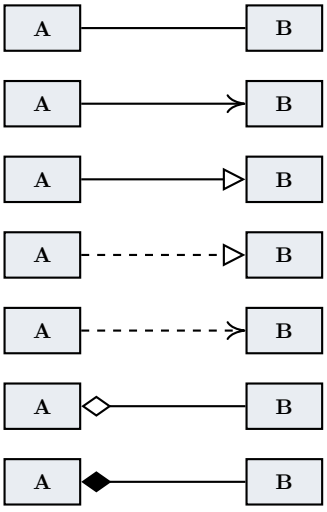


italik ad = **abstract class**

1. bölme: sınıf adı (**tek zorunlu bilgi**)
2. bölme: nitelikler (attributes) ve tipleri
3. bölme: işlemler (operations) ve imzaları

Görünürlük: `-` private, `+` public, `#` protected; **altı çizili** = static (sınıfın bütün nesnelerinde ortak). **Instance** (nesne) gösterimi: adı altı çizilidir ve sınıfını içerebilir (tarif_1974 : TariffSchedule); nitelikler değerleriyle yazılır.

8.2.2 Sınıflar arası ilişkiler



Association
düz çizgi: iki sınıf birbirinden haberdar

Directed (unary) association
açık uçlu ok: A, B'yi bilir; B, A'yı bilmez

Inheritance (generalization)
boş üçgen ata sınıfa doğru: "is-a"

Realization
kesikli çizgi + boş üçgen arayüze doğru: "can-do"

Dependency
kesikli açık ok: "uses-a" (geçici kullanım)

Aggregation
boş elmas **bütün (topluluk)** tarafında: zayıf "has-a"

Composition
dolu elmas **bütün** tarafında: güçlü "has-a", yaşam döngüsü kontrolü

İlişki	Anlamı	Örnek
Association (binary)	Bir sınıfın işini yapabilmek için diğerini bilmesi; mesajlaşma (metotlarına çağrı). İki sınıf da birbirinden haberdar.	Öğrenci – Öğretmen; Customer – Order
Unary / directed association	Tek yönlü haberleşme: A, B'nin metodunu çağırır, B'nin A'dan haberi yok. **Ok çağrılan sınıfa doğrudur.**	OrderDetail → Item
Reflexive association	Bir sınıfın kendisiyle ilişkisi (sınıfın birden çok rolü varsa).	Employee (yönetici) – Employee (çalışan)
Aggregation	"Topluluk–üye" ilişkisi; bir sınıf diğerinin nesnelerinden oluşan bir topluluğu içerir. **Tek sahiplik yok**: parça bütün olmadan yaşayabilir.	Takım ◇– Oyuncu (oyuncu takımsız yaşar); Fakülte ◇– Öğretim üyesi
Composition	Aggregation'ın güçlü hâli: **bütün, parçaların oluşturulmasından ve silinmesinden sorumludur** (lifetime control); parça yalnızca bir bütüne ait olabilir ve bütün olmadan var olamaz.	Ev ◆– Oda; Team ◆– Employee (`new Employee()` / `delete` Team içinde)
Inheritance / generalization	Ata sınıftaki öğelerin alt sınıflarca paylaşılması; "is-a". Ayrıca alt sınıfların belirli özelliklere sahip olduğunun garantisidir.	Gül → Çiçek; Credit, Cash, Check → Payment
Realization	Bir sınıfın bir arayüzü (interface) gerçekleştirmesi; arayüz sadece metot adlarını ve parametrelerini içerir.	Kuş - - ▷ «interface» Uçabilir
Dependency	Bir sınıfın diğerini geçici olarak kullanması (parametre, yerel değişken); diğerindeki değişiklik onu etkileyebilir.	Rapor - - > Yazıcı; Librarian - - «use» > Search

Elmasın yeri

"İçi boş elmas işareti topluluğun olduğu tarafa çizilir, ona bitişik çizilir. Bazı arkadaşlarımız sınavlarda getirip ortaya çiziyorlar veya ters tarafa çizebiliyorlar. Bunlar yanlış." Elmas her zaman **bütünün (collection/whole)** yanındadır. Kalıtım üçgeni her zaman **ata sınıfa** bakar. Directed association oku **çağrılan** sınıfa bakar.

8.2.3 İlişkilerin C# karşılıkları

```
class A { B myB; public void DoSomething() { myB.Service(); } }   // binary association:
class B { A myA; public void Operation()   { myA.DoSomething(); } } //   ikisi de birbirini
    cagirir

class C { B myB; public void Do() { myB.Service(); } }   // unary association: C -> B

class Takim { List<Oyuncu> oyuncular;                    // aggregation:
              public void Ekle(Oyuncu o) { oyuncular.Add(o); } } // oyuncu disarida yaratilir

class Ev { List<Oda> odalar = new List<Oda>();           // composition:
           public Ev() { odalar.Add(new Oda()); } }      // parcayi butun yaratir ve yok eder
```

```
class Gul : Cicek { }                                    // inheritance (is-a)
interface IUcabilir { void Uc(); }
class Kus : IUcabilir { public void Uc() { } }           // realization (can-do)
class Rapor { public void Bas(Yazici y) { y.Yaz(); } }   // dependency (uses-a)
```

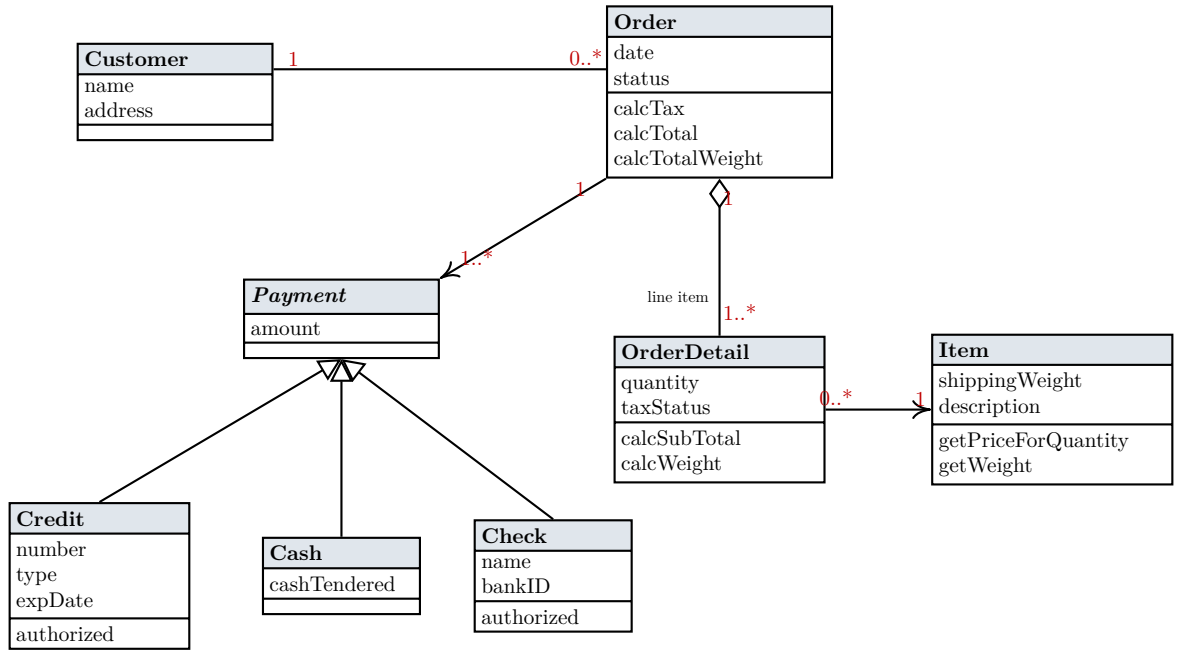
8.2.4 Çokluklar (multiplicities)

İlişki çizgilerinin uçlarına yazılan sayılar, karşı sınıftan **kaç nesne** ile ilişki kurulduğunu gösterir:

Gösterim	Anlamı
`1`	tam olarak bir
`0..1`	sıfır ya da bir
`0..*` ya da `*`	sıfır veya daha fazla (sınırsız, hiç olmayabilir)
`1..*`	en az bir
`n..m`	n ile m arası

İlişki uçlarına **rol adları** da yazılabilir (ör. Team tarafında *group*, Employee tarafında *individual*; Order–OrderDetail'da *line item*). Okuma kuralı: "**Bir** A'nın, **B tarafındaki sayı kadar** B'si vardır."

8.3 UML sunumu sayfa 35: sınıf diyagramını yorumlama



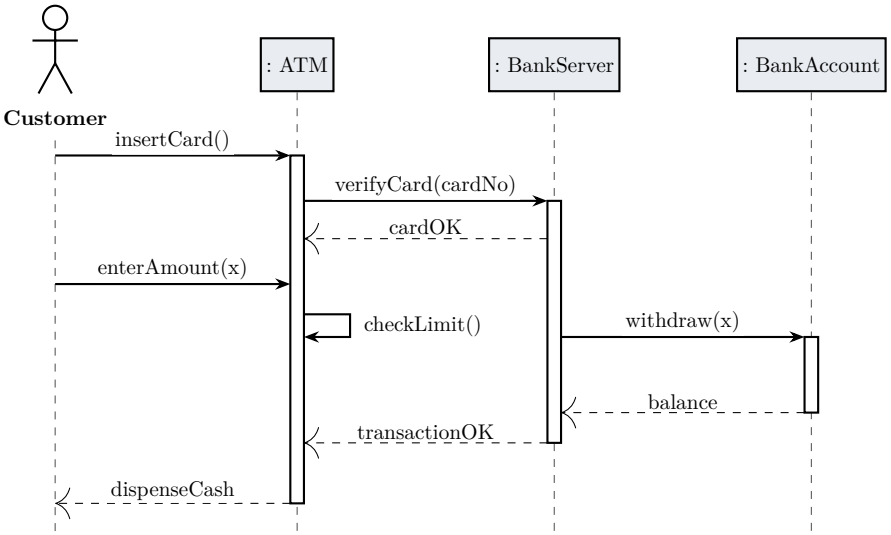
Hocanın derste yaptığı yorum (sınavda beklenen ayrıntı düzeyi)

Önce sınıfları tanıt, sonra ilişkileri, sonra sayıları yorumla. Bu bir sipariş ve ödeme sistemine ait sınıf diyagramıdır.

1. **Customer – Order: binary association** (düz çizgi). Customer ve Order birbirinden haberdardır, birbirlerinin metotlarını çağırabilir. Çokluk: **bir müşterinin sıfır veya daha fazla siparişi** olabilir; her sipariş **tek bir müşteriye** aittir.
2. **Order ◊– OrderDetail: aggregation** (boş elmas Order tarafında). Bir sipariş, sipariş detaylarından (*line item*) oluşan bir topluluktur: **bir siparişin bir veya daha fazla sipariş detayı** vardır ("5 hamburger, 3 salata"). Her detay bir siparişe aittir.

3. **OrderDetail → Item: unary (directed) association**. OrderDetail, Item'ın metotlarını (`getPriceForQuantity`, `getWeight`) çağırır; Item'ın OrderDetail'den haberi yoktur. Çokluk: her sipariş detayı **tam bir** ürüne karşılık gelir; bir ürün **sıfır veya daha fazla** sipariş detayında geçebilir. (Hoca burada "bence 1..* olmalıydı" yorumunu da yaptı: sipariş verilmeden detay olmaz.)
4. **Order → Payment: unary association**. Bir siparişin **bir veya daha fazla ödemesi** olabilir (bir kısmı kart, bir kısmı nakit); her ödeme bir siparişe aittir.
5. **Credit, Cash, Check → Payment: generalization (inheritance)**. Üçü de Payment'ın özel hâlleridir; hepsinde ortak `amount` vardır, ayrıca kendilerine özgü nitelikleri (Credit: number, type, expDate; Check: name, bankID) ve işlevleri (`authorized`) vardır.
6. **Payment adının italik olması**: **abstract class**; gövdesi ata sınıfta yazılmamış üyeler alt sınıflarda tamamlanacaktır; Payment'tan doğrudan nesne üretilmez.

8.4 Sequence diagram notasyonu



- Aktör (use case'ten gelen) ve nesneler (`nesneAdı : SınıfAdı` veya `: SınıfAdı`) en üstte soldan sağa dizilir.
- **Zaman yukarıdan aşağı** akar; nesnelerin altında kesikli **lifeline**, çalıştıkları süre boyunca ince dikdörtgen **activation**.
- **Calls = düz çizgili oklar** (mesaj = çağrılan sınıfın metodu), **returns = kesikli oklar**.
- Nesnenin kendine mesajı (**self call**); nesnenin oluşturulması (`new`) ve yok edilmesi (lifeline sonunda **X**).
- Eğik çizgiler mesaj gecikmesini gösterir (gerçek zamanlı sistemler); mesajlar kesişirse genellikle sorun vardır (*race condition*).
- UML 2'de koşul ve döngüler **combined fragment** ile çizilir: `alt` (if/else), `opt` (if), `loop`.

UML sunumundaki örnekler: *Alarm System* (alarm çalınca zil çalar, ekrana mesaj yazılır, izleme servisine haber verilir) ve *Hotel Reservation*.

8.5 Diğer UML diyagramları

Diyagram	Özeti
Collaboration (communication)	Sequence ile benzer bilgi, ama **dikey zaman ekseni yok**; yerine nesneler arası **bağlantılar** (düz çizgi) ve bağlantılar üzerinde yönlü, numaralı mesajlar. Vurgu statik bağlantılardadır.
Object diagram	Class diyagramının örnekleri ve aralarındaki bağlantılar; belirli bir andaki nesnelerin "fotoğrafı". Biçim: `örnekAdı : SınıfAdı` + nitelik değerleri.
Package diagram	Karmaşık class diyagramlarını mantıksal paketlere gruplar. Paket: üstünde küçük sekmesi olan dikdörtgen; kesikli oklar **bağımlılıktır** (diğer paketteki değişiklik bu paketi değiştirmeye zorlayabilir).
Activity diagram	"Süslü akış diyagramı": tek bir sürecin etkinlik akışı. Köşeleri yuvarlatılmış kutular (etkinlik), **swim lanes** (hangi nesne hangi etkinlikten sorumlu), **baklava** (dallanma), **kalın çubuk** (fork/join: paralel etkinlikler), başlangıç ve bitiş daireleri.
State transition (statechart)	"Süslü DFA": bir nesnenin olası durumları ve durum değişimine yol açan geçişler. Durum = yuvarlatılmış dikdörtgen; geçiş oku üzerinde `Event [Condition] / Action`. Durumun içinde `do/ activity`, `entry/ action`, `exit/ action`. Yerel değişken tutabilir (kütüphane örneği: `Borrow / N = N+1`).
Component diagram	Sistemdeki bileşenler, arayüzleri ve bağımlılıkları; sistemin fiziksel yapısı. Bileşen: sol üstünde iki sekme olan dikdörtgen; arayüz: daire + çizgi (lolipop); bağımlılık: kesikli ok.
Deployment diagram	Kurulmuş sistemin donanım ve yazılım fiziksel mimarisi: **node**'lar (makine/cihaz) ve içlerindeki bileşenler; hangi paket hangi sunucuda (hocanın emlak–banka örneği).

8.6 Kısa tanım soruları

UML A standard graphical modeling language (OMG) for specifying, visualizing, constructing and documenting software; not a process and independent of the implementation language.

Association A relationship between instances of two classes, where one class must know about the other to do its work (solid line; arrow if one-directional).

Aggregation An association where one class belongs to a collection (collection–member); hollow diamond on the collection side; no sole ownership.

Composition A strong form of aggregation with lifetime control: the parts cannot exist without the whole; filled diamond on the whole side.

Inheritance An "is-a" relationship; a hollow triangle pointing to the superclass.

Realization A class implements an interface; dashed line with a hollow triangle pointing to the interface.

Dependency A "uses-a" relationship; a change in one class may affect the other; dashed open arrow.

Multiplicity The number of instances of one class that can be associated with one instance of the other (1, 0..1, *, 1..*, n..m).